

LINKSFOUNDATION.COM



# Earth Observation Meets AI

Foundation Models for EO

LUCA BARCO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

# Training methodologies

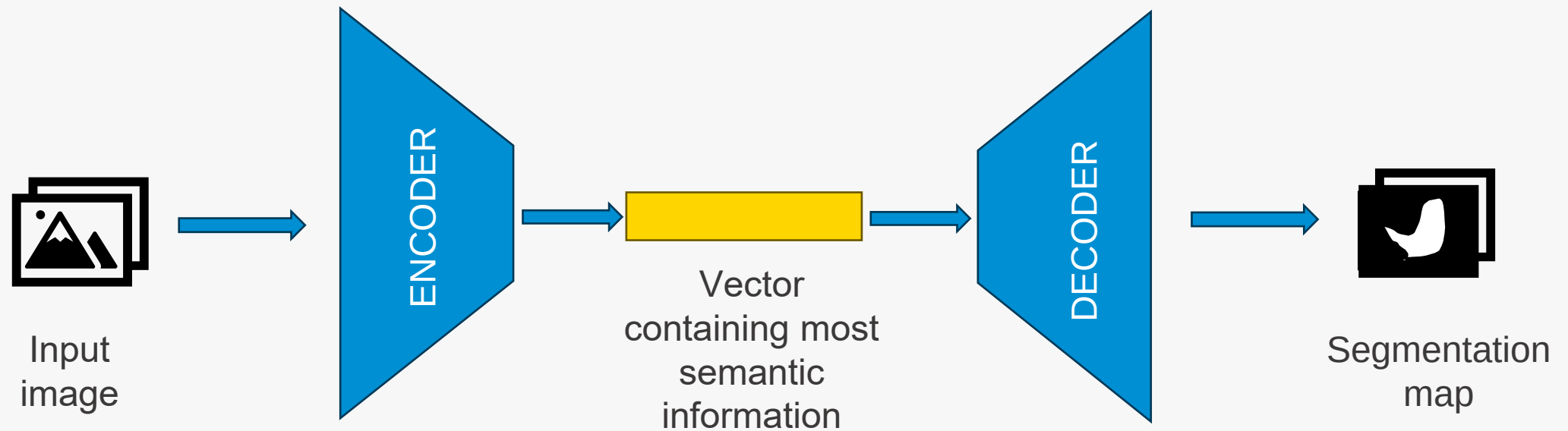
Supervised, Weakly supervised, Unsupervised

- **Supervised learning**
  - **Data** and **labels** are available → model learns on a loss function computed on **labels**
- **Weakly-supervised learning / Adversarial learning**
  - **Data** are available, **labels** are **partially** available → model learns **how to construct labels** using a teacher/student approach.
- **Unsupervised learning**
  - Only **data** are available → model learns **how to represent** data.
- **Multi-Task learning**
  - model must solve **two tasks** at the same time: **the *main* task and the *auxiliary* task**. The latter it is supposed to be beneficial for the main one.

**Very common training strategy for Foundation Models!**

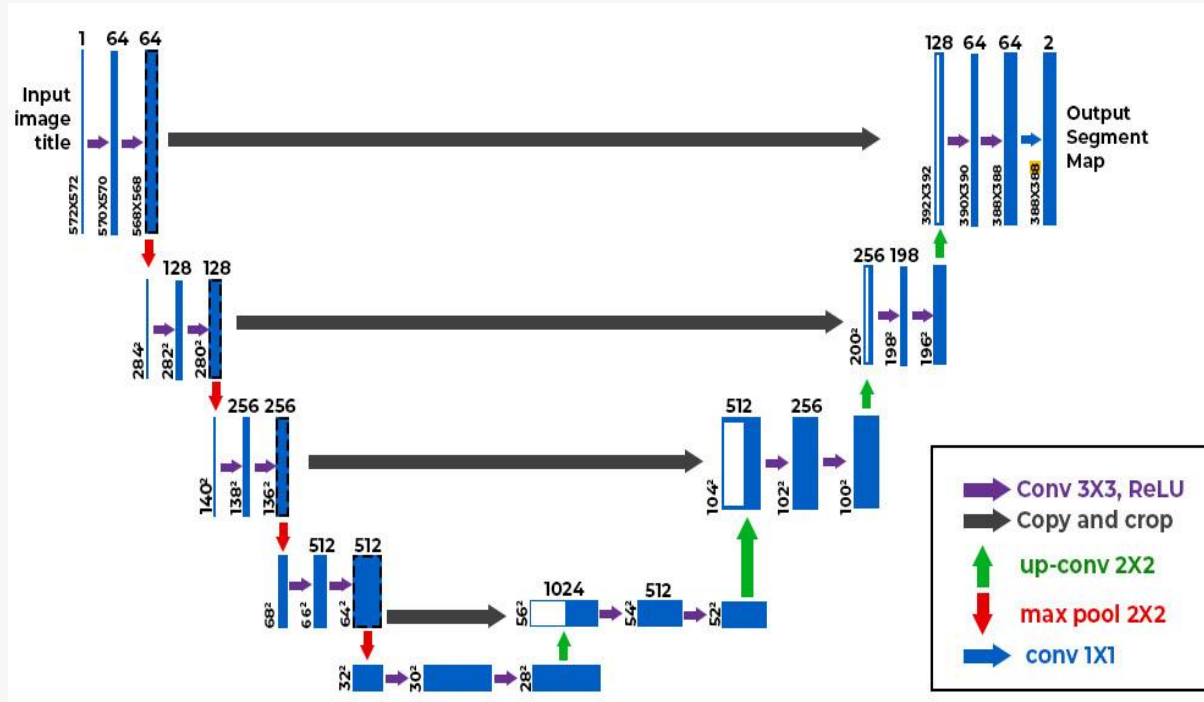
# Semantic Segmentation Models

How a semantic segmentation model looks like



# Example: UNet

How a semantic segmentation model looks like



U-Net Architecture Explained - GeeksforGeeks

**"U" Shape Architecture:** The network has a symmetric U-shaped design with an encoder (contracting path) that shrinks the image to extract features, and a decoder (expansive path) that rebuilds it back to full size.

**Encoder Captures "What":** Uses convolution layers and max pooling to progressively reduce image size while learning increasingly abstract features (e.g., edges → shapes → objects).

**Decoder Reconstructs "Where":** Uses upsampling to gradually restore the original image dimensions and produce a pixel-by-pixel segmentation map.

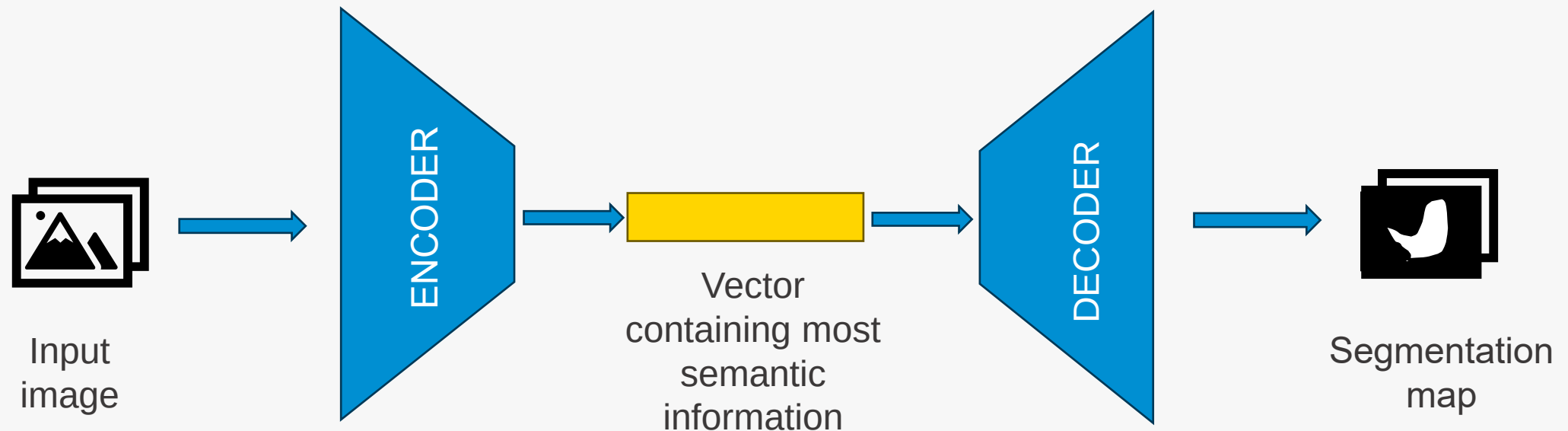
**Skip Connections = Secret Sauce:** Direct connections from encoder to decoder at matching levels preserve fine spatial details that would otherwise be lost during downsampling. This is what makes U-Net so accurate.

**Output = Pixel-Level Classification:** The final result labels every single pixel in the image (e.g., tumor vs. healthy tissue), making it perfect for medical image segmentation where precision matters.

# Foundation Models for EO

## Why?

Let's imagine you want to train a model for a Semantic Segmentation Task.  
This is how the model looks like:



# Foundation Models for EO

Why?

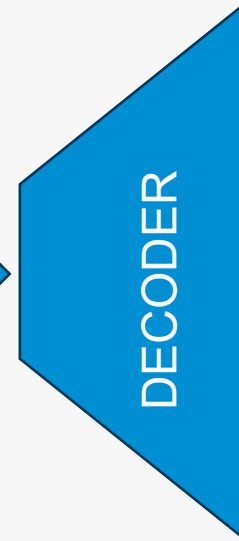
Replace the Encoder with a more powerful backbone  
(more powerful = huge number of parameters, trained on more data, able to manage spectral and temporal information, etc.)



Input image



Vector  
containing most  
semantic  
information



Segmentation  
map

Downstream task

# Foundation Models for EO

Redefining Remote Sensing Analysis

## Key Characteristics

 Pre-trained on massive, diverse satellite imagery datasets

 Transfer learning enables adaptation to downstream tasks

 Multi-temporal, multi-spectral, and often multi-sensor

 Significantly reduce need for labeled data

# Foundation Models for EO

Redefining Remote Sensing Analysis

## Key Characteristics

 Pre-trained on massive, diverse satellite imagery datasets

 Transfer learning enables adaptation to downstream tasks

 Multi-temporal, multi-spectral, and often multi-sensor

 Significantly reduce need for labeled data

## Why now?

 Explosion of satellite data availability

 Petabytes of freely accessible EO archives

 Advances in self-supervised learning techniques

 Democratizing access to powerful EO AI

# Foundation Models Taxonomy

Three main approaches

## Self-supervised learning (SSL)

Learn representations through pretext tasks without labels

Examples: SSL4EO-S12, SatMAE

# Foundation Models Taxonomy

Three main approaches

## Self-supervised learning (SSL)

Learn representations through pretext tasks without labels

Examples: SSL4EO-S12, SatMAE

## Multi-Modal / Multi-Sensor Models

Integrate diverse data sources and modalities for unified representations

Examples: Prithvi-EO, DOFA, Clay, SpectralGPT

# Foundation Models Taxonomy

Three main approaches

## Self-supervised learning (SSL)

Learn representations through pretext tasks without labels

Examples: SSL4EO-S12, SatMAE

## Multi-Modal / Multi-Sensor Models

Integrate diverse data sources and modalities for unified representations

Examples: Prithvi-EO, DOFA, Clay, SpectralGPT

## Large-Scale Unified Models

Massive models trained on billions of patches across sensors and tasks

Examples: Terramind

# Foundation Models for EO

## History

2022

### The SSL Foundation Era

**SSL4EO-S12** - First large-scale benchmark for self-supervised learning in EO

**SatMAE** - Masked autoencoders adapted for satellite imagery with temporal masking

**Established that SSL could rival supervised learning with less labeled data**

# Foundation Models for EO

## History

2022

### The SSL Foundation Era

**SSL4EO-S12** - First large-scale benchmark for self-supervised learning in EO

**SatMAE** - Masked autoencoders adapted for satellite imagery with temporal masking

**Established that SSL could rival supervised learning with less labeled data**

2023

### Multi-Modal & Industry Adoption

**Prithvi-EO** - First major industry-backed open-source FM, HLS dataset

**SpectralGPT** - 3D spatial-spectral architecture for hyperspectral data

**Shift from academic experiments to production-ready, multi-sensor models**

# Foundation Models for EO

## History

2022

### The SSL Foundation Era

**SSL4EO-S12** - First large-scale benchmark for self-supervised learning in EO

**SatMAE** - Masked autoencoders adapted for satellite imagery with temporal masking

**Established that SSL could rival supervised learning with less labeled data**

2023

### Multi-Modal & Industry Adoption

**Prithvi-EO** - First major industry-backed open-source FM, HLS dataset

**SpectralGPT** - 3D spatial-spectral architecture for hyperspectral data

**Shift from academic experiments to production-ready, multi-sensor models**

2024

### Dynamic Architectures & Community Models

**DOFA** - Dynamic weight generation for truly flexible multi-sensor processing

**Clay v1.0** - Community-driven, open governance foundation model

**Focus on flexibility, democratization, and handling any sensor/resolution**

# Foundation Models for EO

## History

2022

### The SSL Foundation Era

**SSL4EO-S12** - First large-scale benchmark for self-supervised learning in EO

**SatMAE** - Masked autoencoders adapted for satellite imagery with temporal masking

Established that SSL could rival supervised learning with less labeled data

2023

### Multi-Modal & Industry Adoption

**Prithvi-EO** - First major industry-backed open-source FM, HLS dataset

**SpectralGPT** - 3D spatial-spectral architecture for hyperspectral data

Shift from academic experiments to production-ready, multi-sensor models

2024

### Dynamic Architectures & Community Models

**DOFA** - Dynamic weight generation for truly flexible multi-sensor processing

**Clay v1.0** - Community-driven, open governance foundation model

Focus on flexibility, democratization, and handling any sensor/resolution

2025

### Scale & Unification

**TerraMind** - Billion-parameter models with emergent capabilities

**SSL4EO-S12 v1.1** - Updated benchmark for next generation models

Scaling laws meet Earth observation: bigger, more unified, more capable

# Self-Supervised Learning

SSL4EO-S12

## SSL4EO-S12

Various Research Groups

### Parameters

~100M (ResNet-50)  
~307M (ViT-L)

### Training data

Sentinel-1 / Sentinel-2  
3mln images

- Benchmark for SSL methods on EO
- Focus on Sentinel-1 and Sentinel-2 multispectral data
- Evaluated on scene classification, semantic segmentation and change detection



SSL viable approach for EO!

## Key Contributions

- Created **large-scale multimodal multitemporal** dataset
- Foundation models available

| Models (params)   | Pretrained on |            |
|-------------------|---------------|------------|
|                   | Sentinel-1    | Sentinel-2 |
| ResNet18 (~11.7M) |               | ✓          |
| ResNet50 (~100M)  | ✓             | ✓          |
| ViT-S (~21M)      | ✓             |            |
| ViT-B (~86M)      | ✓             | ✓          |
| ViT-L (~307M)     | ✓             | ✓          |
| ViT-H (~632M)     | ✓             | ✓          |

# Self-Supervised Learning

SatMAE – Masked Autoencoders for Satellites

## SatMAE

UIUC + Microsoft

### Parameters

~307M (ViT-L)

### Training data

fMoW RGB +  
Sentinel (functional  
Map of the World)

- Adapts **MAE architecture to satellite imagery**
- Temporal and spectral masking strategies
- **Vision Transformer** backbone

## Key Contributions

- Masks patches across **space**, **time**, and **spectrum**
- Learns to reconstruct missing information
- **Handles temporal sequences** natively
- Strong transfer learning to various downstream tasks



Temporal context is crucial  
Multi-temporal masking significantly improves representations

# Multi-Modal/Multi-Sensor

## Spectral-GPT

### Spectral-GPT

Wuhan University

#### Parameters

~100M (Base), 300M  
(Large), 600M (Huge)  
(GPT-style)

#### Training data

Hyperspectral +  
multispectral datasets

- **First** foundation model for spectral RS
- 3D token generation (spatial + spectral)
- Progressive training across resolutions

### Key Contributions

- Treats spectral data as **3D cubes** (height × width × bands)
- **3D tensor tokens** capture spatial-spectral coupling natively
- Progressive training on varying sizes, resolutions, and regions
- Purpose-built for hyperspectral and multispectral imagery



Unlike 2D models that flatten spectral info, SpectralGPT preserves spatial-spectral relationships in 3D

# Multi-Modal/Multi-Sensor

DOFA

## DOFA

Wuhan University

Parameters

~86M (ViT-L)

Training data

Multi-sensors  
(optical, SAR, ...)

- **Single model for multiple sensor types**
- **Dynamic adapter-based architecture**
- Handles varying spectral bands

## Key Contributions

- Input flexibility by **replicating brain plasticity using an hypernetwork** designed to **alter network weights in response to new data modalities**
- Shared backbone with sensor-specific adapters
- Able to work with any number of channels
- Handles heterogeneous data sources elegantly
- Parameter-efficient adaptation to new sensors



One model to rule them all - no need for separate models per sensor

# Multi-Modal/Multi-Sensor

Prithvi-EO

## Prithvi-EO

IBM + NASA

### Parameters

V1.0 ~100M (ViT-B)

V2.0 ~300M (ViT-L)

600M (ViT-H)

- Open-source on Hugging Face
- Harmonized Landsat-Sentinel (HLS) data
- **Temporal encoding for time-series**

### Training data

HSL (1M patches, 6 bands)

## Key Contributions

- Version 1.0 trained on US data only
- Version 2.0 trained on NASA's Harmonized Landsat and Sentinel-2 dataset (30m/pixel resolution) multitemporal samples
- **Excellent** for change detection and temporal analysis
- **MAE with additional positional and temporal encoding**
- Fine-tuned variants available for specific tasks



Good for classification (better than DOFA) and segmentation tasks  
Easily adoptable

# Multi-Modal/Multi-Sensor

Clay

## Clay

Clay Foundation

### Parameters

632M

### Training data

Multi-sensors

(Sentinel, Landsat,  
...) 70M images

- Open-source on GitHub + Hugging Face
- Works with inputs of **any size and any channels**

## Key Contributions

- Input flexibility by **standardizing diverse inputs through a meta-learning stage before processing**
- Exploits geospatial and temporal metadata
- Synthesizes **best** of Prithvi, SatMAE, ScaleMAE, DOFA, and SpectralGPT



Good for classification and segmentation tasks  
Easily adoptable

# Large-Scale Unified Models

TerraMind

## Terramind

Various Tech Giants

Parameters

1B+

Training data

Billions of patches,  
multi-sensor

- **Largest** EO foundation model to date
- **Unified architecture** for diverse tasks
- **State-of-the-art performance across benchmarks**

## Key Contributions

- Emergent capabilities from massive scale
- Handles **multiple sensors, resolutions, and tasks** simultaneously
- **Strong zero-shot and few-shot learning abilities**
- Captures **global patterns** and **local details**



Good for classification (better than DOFA) and segmentation tasks  
Easily adoptable

# Model Comparison

## Key Differences and Trade-offs

| Model       | Approach     | Scale            | Sensors                       | Open      | Best for                      |
|-------------|--------------|------------------|-------------------------------|-----------|-------------------------------|
| SSL4EO-S12  | SSL          | Medium           | Sentinel-2                    | ✓ Yes     | SSL research benchmark        |
| SatMAE      | SSL          | Medium           | Multi-temporal optical        | ✓ Yes     | Temporal analysis             |
| Prithvi-EO  | Multi-Sensor | Medium           | HLS (L8 + S2)                 | ✓ Yes     | Change detection, time-series |
| SpectralGPT | Multi-Sensor | Large (600M)     | Hyperspectral + Multispectral | ✓ Yes     | Spectral analysis, 3D data    |
| DOFA        | Multi-Sensor | Medium           | Multi-sensor (optical, SAR)   | ✓ Yes     | Sensor fusion                 |
| Clay        | Multi-Sensor | Variable         | Multi-sensor                  | ✓ Yes     | Community projects            |
| TerraMind   | Large-Scale  | Very Large (1B+) | Multi-sensor, global          | ⚠ Limited | Production systems, SOTA      |

LINKSFOUNDATION.COM



# Earth Observation Meets AI

TorchGeo + Terratorch

LUCA BARCO

APPLIED DATA SCIENCE PROJECT, A.Y. 2025/2026

# TorchGeo

- **TorchGeo** is a PyTorch library designed for machine learning experts to work with geospatial data, and for remote sensing experts to **explore machine learning solutions**
- Provides:
  - **Datasets**
  - **Samplers**
  - **Transforms**
  - **Pre-trained models**
- Everything is compatible with Pytorch modules → it can be used with wrapper libraries like Pytorch Lightning



# Datasets

TorchGeo has two types of datasets (instances of Pytorch Datasets):

- **Geospatial Datasets**: designed for data with **geospatial information** like latitude, longitude, CRS, time (if available). These information are exploited to perform **queries** on specific geographic areas or on specific date (i.e. Turin area in July 2021)
  - **Indexed by a spatiotemporal bounding box**
  - Two or more datasets covering a different geographic extent can be intelligently combined using
    - IntersectionDataset : queries must be present in both datasets
    - UnionDataset: queries must be present in at least one of the datasets
  - List of available datasets here: [torchgeo.datasets — torchgeo 0.8.0.dev0 documentation](#)
- **Non-geospatial Datasets**: designed for data **without** geospatial information
  - Can be combined together using a ConcatDataset (just put together data)
  - List of available datasets here: [torchgeo.datasets — torchgeo 0.8.0.dev0 documentation](#)
- For some datasets, also DataModules (instances of Pytorch Lightning Data Modules) are available: [torchgeo.datamodules — torchgeo 0.8.0.dev0 documentation](#)

# GeoDataset vs Non GeoDataset

## Main differences

### Non GeoDataset

Traditional Image Classification

Index-based: dataset[id]

|      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|
| [0]  | [1]  | [2]  | [3]  | [4]  | [5]  | [6]  |
| [7]  | [8]  | [9]  | [10] | [11] | [12] | [13] |
| [14] | [15] | [16] | [17] | [18] | [19] | [20] |
| [21] | [22] | [23] | [24] | [25] | [26] | [27] |



#### Integer Indexing

Access by index: dataset[0], dataset[1], ...



#### Fixed labels

Each image has a pre-assigned class label



#### Discrete samples

Each sample is independent



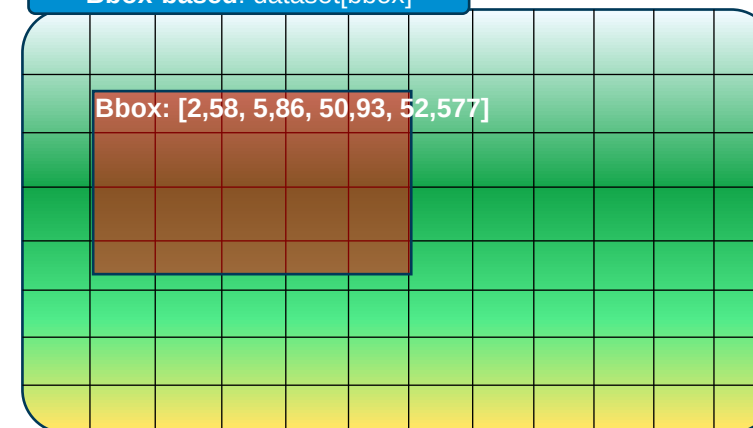
#### Random Sampling

Use standard PyTorch RandomSampler

### GeoDataset

Geospatial data with coordinates

Bbox-based: dataset[bbox]



#### Bounding Box Query

Access by coordinates: dataset[bbox]



#### Spatial Indexing

Built-in R-tree for efficient spatial queries



#### CRS & Resolution

Coordinate reference system and pixel resolution metadata



#### Dataset Fusion

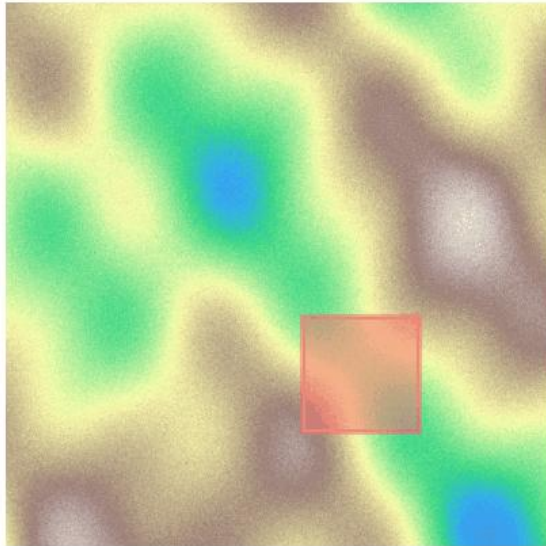
Combine multiple datasets using spatial intersection/union

# Samplers

- **Samplers** are used to index dataset **retrieving a single query at a time**.
  - **Geospatial** datasets → **Geospatial queries** → **GeoSampler**
  - **Non-geospatial** datasets → **Integer queries** (standard **PyTorch samplers** are good)
- **Geospatial datasets**
  - have a bounding box definition used for indexing
  - **Different sampling strategies**
    - **RandomGeoSampler**: samples elements from a region of interest randomly (may overlap)
    - **GridGeoSampler**: samples elements using a grid (no overlap)
    - **PreChippedGeoSampler**: samples entire files at a time
- For all samplers, the Batch versions are available [... torchgeo.samplers — torchgeo 0.8.0.dev0 documentation](https://pytorch-geo.github.io/torchgeo/0.8.0.dev0/)

# Random VS Grid Geo Sampler

**Random Geo Sampler**

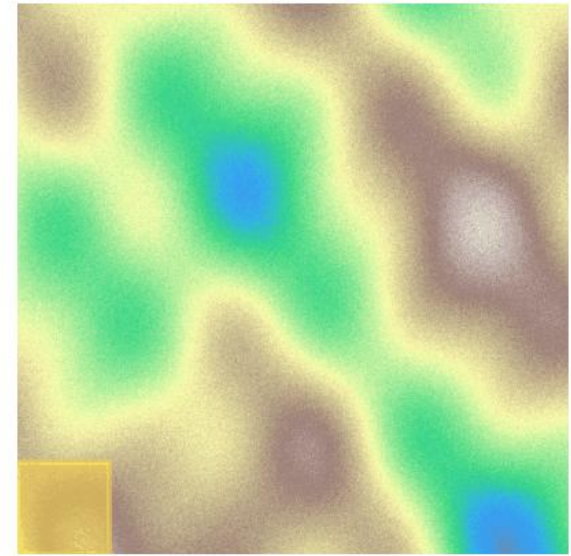


Randomly samples patches from the dataset.  
Ideal for training with diverse samples.

Samples: 1 / 50  
Patch Size: 128px  
Coverage: 4.6%

Good for Training

**Grid Geo Sampler**



Samples in regular grid pattern.  
Perfect for inference across entire images.

Samples: 1 / 36  
Patch Size: 100px  
Coverage: 2.8%

Mandatory for Testing

# Transforms

- Use [Kornia](#) library
  - GPU-accelerated transforms, works with **multispectral bands** (not just RGB).
  - Provides **common data augmentations**: flips, rotations, elastic warps, random crops.
  - Handles **geospatial specifics**:
    - Seasonal/temporal shifts
    - Radiometric corrections
    - Cloud cover simulation
- TorchGeo → **Spectral indices support**: computes indices directly from multispectral bands and **adds them as new channels**.
  - Examples:
    - **NDVI** – vegetation health
    - **NDWI** – water presence
    - **NDBI** – built-up areas
    - **NDSI** – snow cover
    - ... [torchgeo.transforms — torchgeo 0.8.0.dev0 documentation](#)

# Models

- Predefined deep learning architectures **adapted** to remote sensing tasks.
- Implemented with PyTorch / PyTorch Lightning
- Drop-in integration with PyTorch Lightning training loops
- **Pretraining helps** when labeled data are scarce

Examples of available models:

**UNet** (semantic segmentation)

**ResNet** (classification, can be used as backbone for segmentation)

**SwinTransformer** (classification, can be used as backbone for segmentation)

**YOLO** (object detection)

... and more [torchgeo.models — torchgeo 0.8.0.dev0 documentation](https://pytorch-geo.github.io/torchgeo/0.8.0.dev0/models/)

# Trainers

- Predefined **Pytorch Lightning Trainers** for **specific tasks**
  - **ChangeDetectionTask**
  - **ClassificationTask**
  - **InstanceSegmentationTask**
  - **RegressionTask**
  - **PixelWiseRegressionTask**
  - **SemanticSegmentationTask**
- ... [torchgeo.trainers — torchgeo 0.8.0.dev0 documentation](#)

# TorchGeo

Hands-on session

# Terratorch

a PyTorch domain library based on **PyTorch Lightning** + **TorchGeo** domain library for geospatial data.

You can use it as

- **Python package** (as a common library)
- **CLI command** writing **YAML** configuration files

```
from terratorch.tasks import PixelwiseRegressionTask

model_args = dict(
    backbone="prithvi_eo_v2_300",
    backbone_pretrained=True,
    backbone_num_frames=1,
    backbone_bands=[
        "BLUE",
        "GREEN",
        "RED",
        "NIR_NARROW",
        "SWIR_1",
        "SWIR_2",
    ],
    necks=[
        {"name": "SelectIndices", "indices": [5, 11, 17, 23]},
        {"name": "ReshapeTokensToImage"},
        {"name": "LearnedInterpolateToPyramidal"}
    ],
    decoder="UNetDecoder",
    decoder_channels=[512, 256, 128, 64],
    head_dropout=0.1,
)

task = PixelwiseRegressionTask(
    model_factory="EncoderDecoderFactory",
    model_args=model_args,
    loss="rmse",
    lr=1e-4,
    ignore_index=-1,
    optimizer="AdamW",
    optimizer_hparams={"weight_decay": 0.05},
)

# Pass this LightningModule to a Lightning Trainer, together with some LightningDataModule
...
```

```
model:
  class_path: terratorch.tasks.PixelwiseRegressionTask
  init_args:
    model_args:
      decoder: UperNetDecoder
      pretrained: false
      backbone: prithvi_eo_v2_600
      backbone_drop_path_rate: 0.3
      backbone_window_size: 8
      decoder_channels: 64
      num_frames: 1
      in_channels: 6
      bands:
        - BLUE
        - GREEN
        - RED
        - NIR_NARROW
        - SWIR_1
        - SWIR_2
      head_dropout: 0.5708022831486758
      head_final_act: torch.nn.ReLU
      head_learned_upscale_layers: 2
  loss: rmse
  ignore_index: -1
  freeze_backbone: true
  freeze_decoder: false
  model_factory: PrithviModelFactory
  tiled_inference_parameters:
    h_crop: 224
    h_stride: 192
    w_crop: 224
    w_stride: 192
```

```
terratorch fit --config <path_to_config_file>
```

# Terratorch

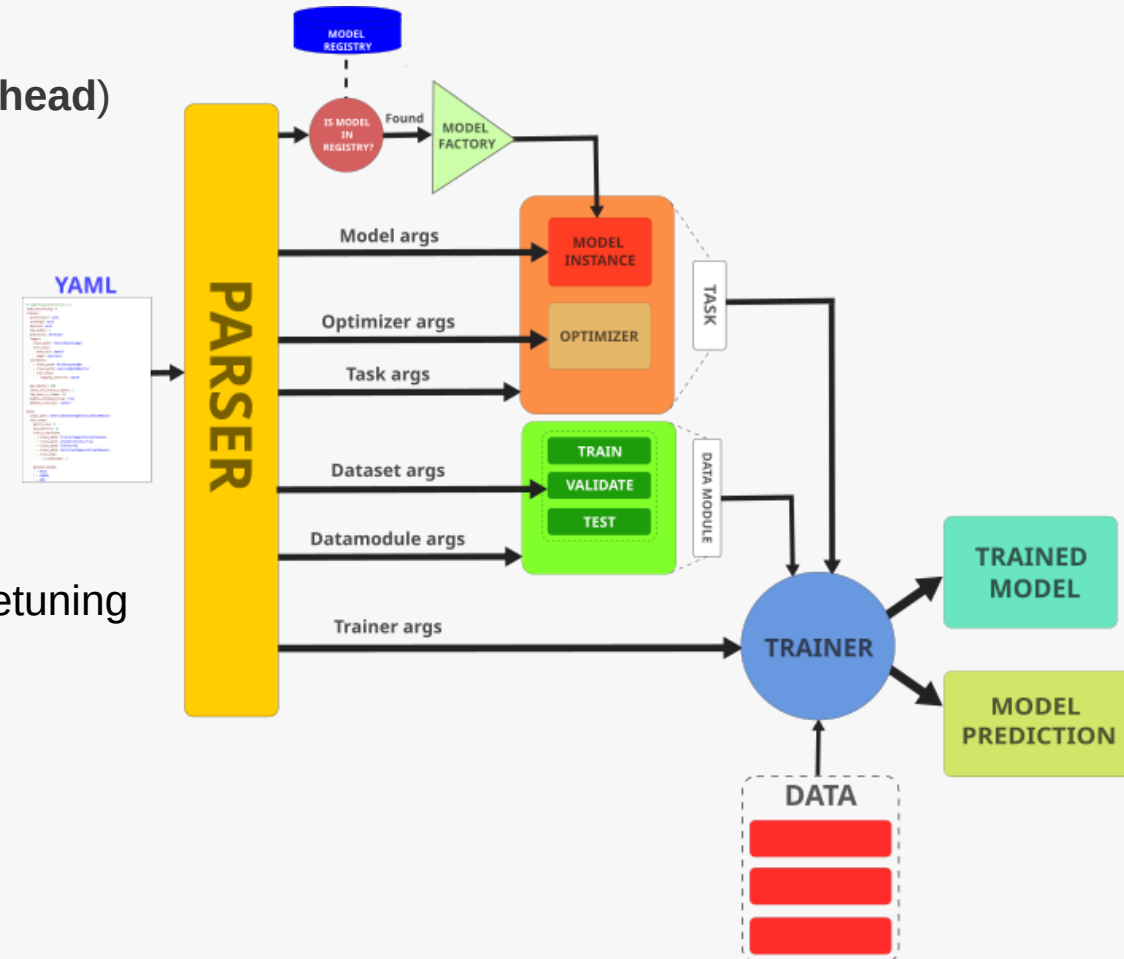
a PyTorch domain library based on **PyTorch Lightning** + **TorchGeo** domain library for geospatial data.

Basic elements:

- **TASK** → **coordinator** for training + inference (Lightning Module)
- **MODEL** → Internally divided into **backbone** + **neck** + **decoder** + **head**)
- **LOSS** → **handler** for loss computation
- **DATA** → **build your own data logic using TorchGeo**

You can use Terratorch's model collection as

- **Backbone factory** (only for pretrained FM encoders)
- **Full model factory** (Pretrained encoder + decoder)
- **LightningModule factory** (Pretrained models + code for finetuning a complete task, i.e. for semantic segmentation)
- You can **mix** backbones, necks, decoders, heads
- + **you can add your own model!**



# Terratorch available Foundation Models

| Model                            | Description                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Prithvi                          | Vision transformer pretrained on multispectral EO data; strong for <b>classification</b> and <b>segmentation</b> tasks.                         |
| TerraMind                        | <b>Self-supervised</b> transformer for Earth observation; captures spatial and spectral correlations across multiple sensors.                   |
| SatMAE                           | <b>Masked autoencoder</b> for satellite imagery; learns representations by reconstructing masked patches, effective for data-scarce tasks.      |
| ScaleMAE                         | Variant of SatMAE designed to handle <b>multi-scale satellite imagery</b> ; robust for high-resolution EO data.                                 |
| Satlas (TorchGeo)                | Foundation model pretrained on <b>large-scale Sentinel-2 imagery</b> ; supports segmentation and classification.                                |
| DOFA (TorchGeo)                  | Self-supervised EO model focused on <b>domain adaptation</b> ; good for cross-sensor generalization.                                            |
| SSL4EO-L / SSL4EO-S12 (TorchGeo) | Self-supervised learning on <b>Landsat (L)</b> or <b>Sentinel-2 (S12)</b> datasets; captures spectral and spatial patterns without labels.      |
| Clay                             | Convolutional and transformer <b>hybrid backbone</b> pretrained on global EO datasets; designed for <b>multisource and multitemporal data</b> . |



integration with **timm** and **SMP** model libraries

# Terratorch

Hands-on session